

Linked list

Linked lists are a versatile data structure that consist of nodes, where each node contains data and a pointer (or reference) to the next node in the sequence. Unlike arrays, linked lists do not have a fixed size and can easily grow and shrink. Here are the possible operations you can perform on linked lists:

1. **Insertion**

- ****At the Beginning****: Insert a new node at the start of the list.

- ****Example****:

```
```cpp
void insertAtBeginning(Node*& head, int newData) {
 Node* newNode = new Node(newData);
 newNode->next = head;
 head = newNode;
}
```
```

- ****At the End****: Insert a new node at the end of the list.

- ****Example****:

```
```cpp
void insertAtEnd(Node*& head, int newData) {
 Node* newNode = new Node(newData);
 if (head == nullptr) {
 head = newNode;
 return;
 }
 Node* temp = head;
 while (temp->next != nullptr) {
 temp = temp->next;
 }
 temp->next = newNode;
}
```
```

- ****At a Specific Position****: Insert a new node after a given node.

- ****Example****:

```
```cpp
void insertAfter(Node* prevNode, int newData) {
 if (prevNode == nullptr) return;
 Node* newNode = new Node(newData);
 newNode->next = prevNode->next;
 prevNode->next = newNode;
}
```
```

2. **Deletion**

- ****Delete a Node by Value****: Remove the first occurrence of a node containing a specific value.

- ****Example****:

```
```cpp
void deleteNode(Node*& head, int key) {
 Node* temp = head, *prev = nullptr;
 if (temp != nullptr && temp->data == key) {
 head = temp->next;
 delete temp;
 return;
 }
 while (temp != nullptr && temp->data != key) {
 prev = temp;
 temp = temp->next;
 }
 if (temp == nullptr) return; // Key not found
 prev->next = temp->next;
 delete temp;
}
```
```

- ****Delete a Node at a Specific Position****: Remove a node at a given index.

- ****Example****:

```
```cpp
void deleteAtPosition(Node*& head, int position) {
 if (head == nullptr) return;
 Node* temp = head;
 if (position == 0) {
 head = temp->next; // Change head
 delete temp;
 return;
 }
 for (int i = 0; temp != nullptr && i < position - 1; i++) {
 temp = temp->next;
 }
 if (temp == nullptr || temp->next == nullptr) return;
 Node* next = temp->next->next;
 delete temp->next; // Free memory
 temp->next = next; // Unlink the deleted node
}
```
```

3. ****Traversal****

- ****Print Elements****: Iterate through the list and print all elements.

- ****Example****:

```
```cpp
void printList(Node* node) {
```

```

 while (node != nullptr) {
 cout << node->data << " ";
 node = node->next;
 }
 }
 ...

```

#### ### 4. **\*\*Searching\*\***

- **\*\*Search for a Value\*\***: Find if a specific value exists in the list.

- **\*\*Example\*\***:

```

```cpp
bool search(Node* head, int key) {
    Node* current = head;
    while (current != nullptr) {
        if (current->data == key) return true;
        current = current->next;
    }
    return false;
}
...

```

5. ****Reversal****

- ****Reverse the Linked List****: Reverse the order of nodes in the list.

- ****Example****:

```

```cpp
void reverse(Node*& head) {
 Node* prev = nullptr;
 Node* current = head;
 Node* next = nullptr;
 while (current != nullptr) {
 next = current->next; // Store next node
 current->next = prev; // Reverse current node
 prev = current; // Move pointers forward
 current = next;
 }
 head = prev; // Update head
}
...

```

#### ### 6. **\*\*Counting Nodes\*\***

- **\*\*Count the Number of Nodes\*\***: Calculate the total number of nodes in the list.

- **\*\*Example\*\***:

```

```cpp
int countNodes(Node* head) {
    int count = 0;
    while (head != nullptr) {
        count++;
    }
}
...

```

```

        head = head->next;
    }
    return count;
}
```

```

### ### 7. \*\*Clearing the List\*\*

- **\*\*Delete All Nodes\*\***: Free memory for all nodes in the list.
- **\*\*Example\*\***:

```

```cpp
void clearList(Node*& head) {
    Node* current = head;
    Node* nextNode = nullptr;
    while (current != nullptr) {
        nextNode = current->next;
        delete current; // Free memory
        current = nextNode; // Move to next node
    }
    head = nullptr; // Set head to nullptr
}
```

```

### ### Example Structure for a Node

Here's a simple example of how you might define a node in a linked list in C++:

```

```cpp
struct Node {
    int data;
    Node* next;

    Node(int value) {
        data = value;
        next = nullptr;
    }
};
```

```

### ### Summary

These operations allow for a comprehensive manipulation of linked lists, enabling a wide range of applications in algorithms, data storage, and memory management. If you have specific examples or additional questions regarding linked lists, feel free to ask!